

KMP算法（Knuth-Morris-Pratt算法）是一种高效的字符串匹配算法，其核心原理在于利用匹配失败后的信息，通过预处理模式串来避免主串指针的回溯，从而将匹配时间复杂度从暴力算法的 $O(m \times n)$ 优化至 $O(m+n)$ ¹。

为了透彻理解其原理，我们可以遵循以下逻辑展开：
一、从暴力匹配的局限性看KMP的优化动机。

传统的暴力匹配算法在发现字符不匹配时，会将主串的指针回退到本次匹配起始位置的下一位，并将模式串指针重置为开头，从头开始比较。

例如，在主串 "ABABCABABC" 中查找模式串 "ABABD"，当前四个字符 "ABAB" 匹配成功，第五个字符失配时，暴力算法会让主串从第二个字符 'B' 开始，重新与模式串的第一个字符 'A' 比较。这种做法没有利用已经匹配成功的 "ABAB" 这个信息，导致了大量重复且无效的比较。

KMP算法的提出正是为了解决这一问题。它的基本思想是：当出现字符串不匹配时，可以知道一部分已经匹配的文本内容，利用这些信息避免从头再去匹配。

KMP算法高效的关键在于它预先分析了模式串自身的结构，其分析结果存储在一个被称为 部分匹配表（Partial Match Table, PMT） 或 Next数组 的数组中。要理解这个表，必须先理解两个概念：

- 前缀：指除最后一个字符外，一个字符串的全部头部组合。
- 后缀：指除第一个字符外，一个字符串的全部尾部组合。

以模式串 "ABABC" 为例：

- 对于子串 "ABAB"，
其前缀有 {A, AB, ABA}，
后缀有 {B, AB, BAB}。

两者的公共元素是 "AB"，长度为2。

- 这个 最长公共前后缀的长度 就是部分匹配表在该位置存储的值 2 。

部分匹配表（PMT/Next数组）的定义是：对于模式串P，next[i] 表示 P[0...i] 这个子串中，最长的、相等的前缀和后缀的长度。

例如，模式串 "ABABACA" 的next数组计算如下表所示

索引(i)	0	1	2	3	4	5	6
字符	A	B	A	B	A	C	A
next[i]	0	0	1	2	3	0	1

这个表的核心作用在于：当在模式串的第 i 位与主串发生失配时，模式串可以直接向右滑动 (已匹配字符数 - next[i-1]) 位，而不是仅仅移动一位 ¹。因为 next[i-1] 告诉我们，已匹配成功的子串末尾有一部分（后缀）和开头的一部分（前缀）是相同的，那么我们就可以将前缀对齐到刚才后缀的位置，从而跳过那些已知肯定匹配的部分，从新的位置开始比较。

三、算法实现的两大步骤 KMP算法的实现清晰地分为两个阶段，其代码结构高度相似。

1. 预处理：构建Next数组

构建Next数组的过程，可以看作模式串自我匹配的过程。我们使用两个指针 i 和 j ，其中 j 指向前缀的末尾（也代表了当前最长公共前后缀的长度）， i 指向后缀的末尾。其核心逻辑是一个动态规划的过程：

- 初始化： $\text{next}[0] = 0$ (或 -1 ，取决于实现约定)， $j = 0$ 。
- 循环 (i 从 1 开始)：
 - 若 $P[i] == P[j]$ ，则找到更长的公共前后缀， $j++$ ，并令 $\text{next}[i] = j$ 。
 - 若 $P[i] != P[j]$ ，则失配。此时不能直接重置 $j=0$ ，因为可能存在于更短的前缀匹配。

根据KMP思想，我们需要在“已匹配”的前缀串中，继续查找其最长公共前后缀，即令 $j = \text{next}[j-1]$ 并继续尝试匹配，直到 j 回溯到 0。

2. 匹配：利用Next数组搜索

匹配过程与构建Next数组的过程几乎一致，可以理解为模式串与主串的匹配。使用指针 i 遍历主串，指针 j 指向模式串。

初始化： $i = 0, j = 0$ 。

- 循环 (i 未到主串末尾)：
 - 若 $T[i] == P[j]$ ，则 $i++, j++$ 。
 - 若 $T[i] != P[j]$ ，则失配：

■ 如果 $j > 0$ ，根据Next数组，将模式串指针回退到 $j = \text{next}[j-1]$ ，主串指针 i 不动。这是KMP避免主串回溯的关键。

■ 如果 $j == 0$ ，说明模式串第一个字符就不匹配，则直接 $i++$ ，主串后移一位。

• 终止：当 j 等于模式串长度时，说明找到完全匹配，返回 $i - j$ 即为起始位置。